

Assignment 10: Data Scraping

Isabella Yang

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on data scraping.

Directions

1. Rename this file `<FirstLast>_A10_DataScraping.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:
 - Load the packages `tidyverse`, `rvest`, and any others you end up using.
 - Check your working directory

```
#1
library(tidyverse)
library(lubridate)
library(rvest)
library(here); here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

2. We will be scraping data from the NC DEQs Local Water Supply Planning website, specifically the Durham’s 2024 Municipal Local Water Supply Plan (LWSP):
 - Navigate to <https://www.ncwater.org/WUDC/app/LWSP/search.php>
 - Scroll down and select the LWSP link next to Durham Municipality.
 - Note the web address: <https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024>

Indicate this website as the as the URL to be scraped. (In other words, read the contents into an `rvest` webpage object.)

#2

```
webpage_durham <- read_html('https://www.ncwater.org/WUDC/app/LWSP/report.php?psid=03-32-010&year=2024')
webpage_durham
```

```
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equiv= ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...
```

3. The data we want to collect are listed below:

- From the “1. System Information” section:
 - Water system name
 - PWSID
 - Ownership
- From the “3. Water Supply Sources” section:
 - Maximum Day Use (MGD) - for each month

In the code chunk below scrape these values, assigning them to four separate variables.

HINT: The first value should be “Durham”, the second “03-32-010”, the third “Municipality”, and the last should be a vector of 12 numeric values (represented as strings)“.

#3

```
city_name <- webpage_durham %>%
  html_nodes("div+ table tr:nth-child(1) td:nth-child(2)") %>%
  html_text()
city_name
```

```
## [1] "Durham"
```

```
PWSID <- webpage_durham %>%
  html_nodes("td tr:nth-child(1) td:nth-child(5)") %>%
  html_text()
PWSID
```

```
## [1] "03-32-010"
```

```
Ownership <- webpage_durham %>%
  html_nodes("div+ table tr:nth-child(2) td:nth-child(4)") %>%
  html_text()
Ownership
```

```
## [1] "Municipality"
```

```
MGD <- webpage_durham %>%
  html_nodes("th~ td+ td , th~ td+ td") %>%
  html_text()
MGD
```

```
## [1] "34.5000" "36.0600" "37.3300" "32.1000" "46.6500" "37.3600" "38.2000"
## [8] "41.9000" "36.5800" "36.7300" "42.9600" "34.4500"
```

4. Convert your scraped data into a dataframe. This dataframe should have a column for each of the 4 variables scraped and a row for the month corresponding to the withdrawal data. Also add a Date column that includes your month and year in data format. (Feel free to add a Year column too, if you wish.)

TIP: Use `rep()` to repeat a value when creating a dataframe.

NOTE: It's likely you won't be able to scrape the monthly withdrawal data in chronological order. You can overcome this by creating a month column manually assigning values in the order the data are scraped: "Jan", "May", "Sept", "Feb", etc...

5. Create a line plot of the maximum daily withdrawals across the months for 2024, making sure, the months are presented in proper sequence.

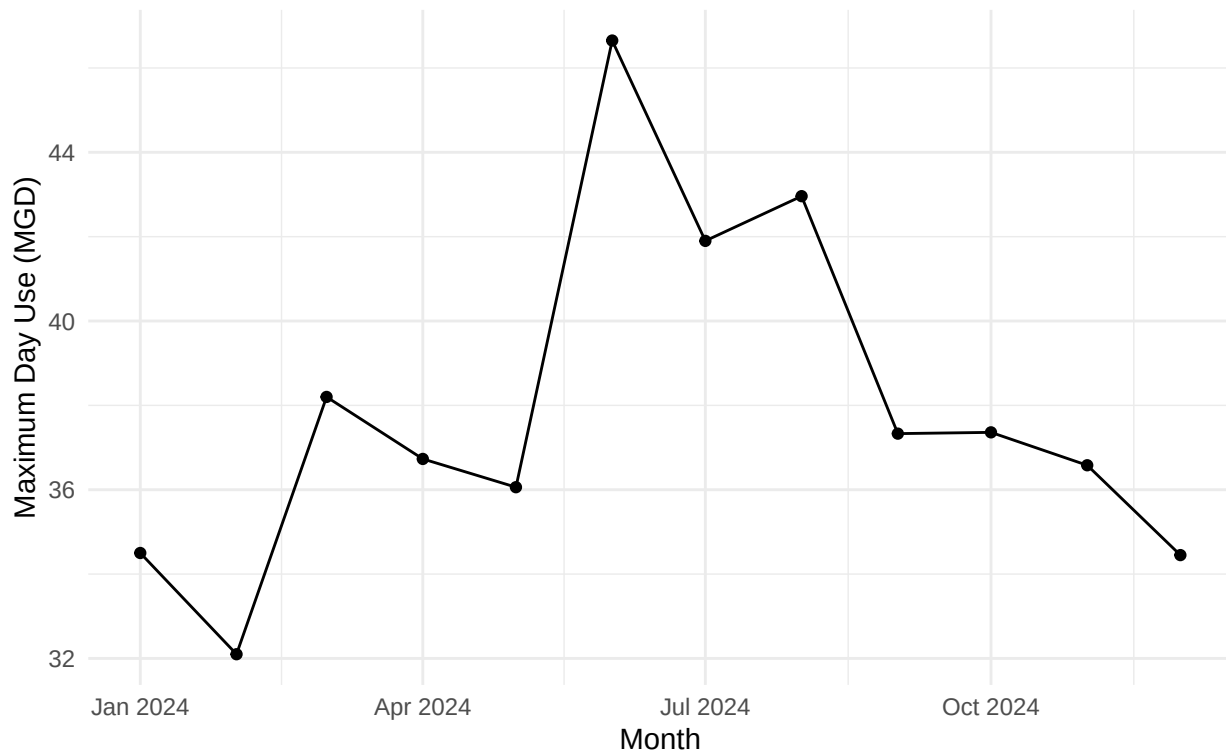
```
#4
#Create a dataframe of withdrawals
df_durham <- data.frame("Month" = c("Jan", "May", "Sep", "Feb", "June", "Oct",
  "Mar", "Jul", "Nov", "Apr", "Aug", "Dec"),
  "Year" = rep(2024, 12),
  "Maximum_Day_Use" = as.numeric(MGD))

#Modify the dataframe to include the facility name and type as well as the date (as date object)
df_durham <- df_durham %>%
  mutate(Date = my(paste(Month, Year))) %>%
  arrange(Date)

#5
#Plot
ggplot(df_durham, aes(x = Date, y = Maximum_Day_Use)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Maximum Day Use (MGD) in 2024",
    subtitle = "Durham Municipal LWSP",
    x = "Month",
    y = "Maximum Day Use (MGD)") +
  theme_minimal()
```

Maximum Day Use (MGD) in 2024

Durham Municipal LWSP



6. Note that the PWSID and the year appear in the web address for the page we scraped. Construct a function with two inputs - “PWSID” and “year” - that:

- Creates a URL pointing to the LWSP for that PWSID for the given year
- Creates a website object and scrapes the data from that object (just as you did above)
- Constructs a dataframe from the scraped data, mostly as you did above, but includes the PWSID and year provided as function inputs in the dataframe.
- Returns the dataframe as the function’s output

```
#6.
scrape_it <- function(PWSID, year){
  webpage_durham <- read_html(paste0('https://www.ncwater.org/WUDC/app/LWSP/report.php?psid=',
                                     PWSID, '&year=', year))
  city_name <- 'div+ table tr:nth-child(1) td:nth-child(2)'
  PWSID <- 'td tr:nth-child(1) td:nth-child(5)'
  Ownership <- 'div+ table tr:nth-child(2) td:nth-child(4)'
  MGD <- 'th~ td+ td , th~ td+ td'
  city_name <- webpage_durham %>% html_nodes(city_name) %>% html_text()
  PWSID <- webpage_durham %>% html_nodes(PWSID) %>% html_text()
  Ownership <- webpage_durham %>% html_nodes(Ownership) %>% html_text()
  MGD <- webpage_durham %>% html_nodes(MGD) %>% html_text()
  df_durham <- data.frame("Month" = c("Jan", "May", "Sep", "Feb", "June", "Oct",
                                       "Mar", "Jul", "Nov", "Apr", "Aug", "Dec"),
                         "Year" = rep(year, 12),
                         "Maximum_Day_Use" = as.numeric(MGD),
```

```

    "Water_System" = rep(city_name, 12),
    "PWSID" = rep(PWSID, 12),
    "Ownership" = rep(Ownership, 12)) %>%
  mutate(Date = my(paste(Month, Year))) %>%
  arrange(Date)
  return(df_durham)
}

```

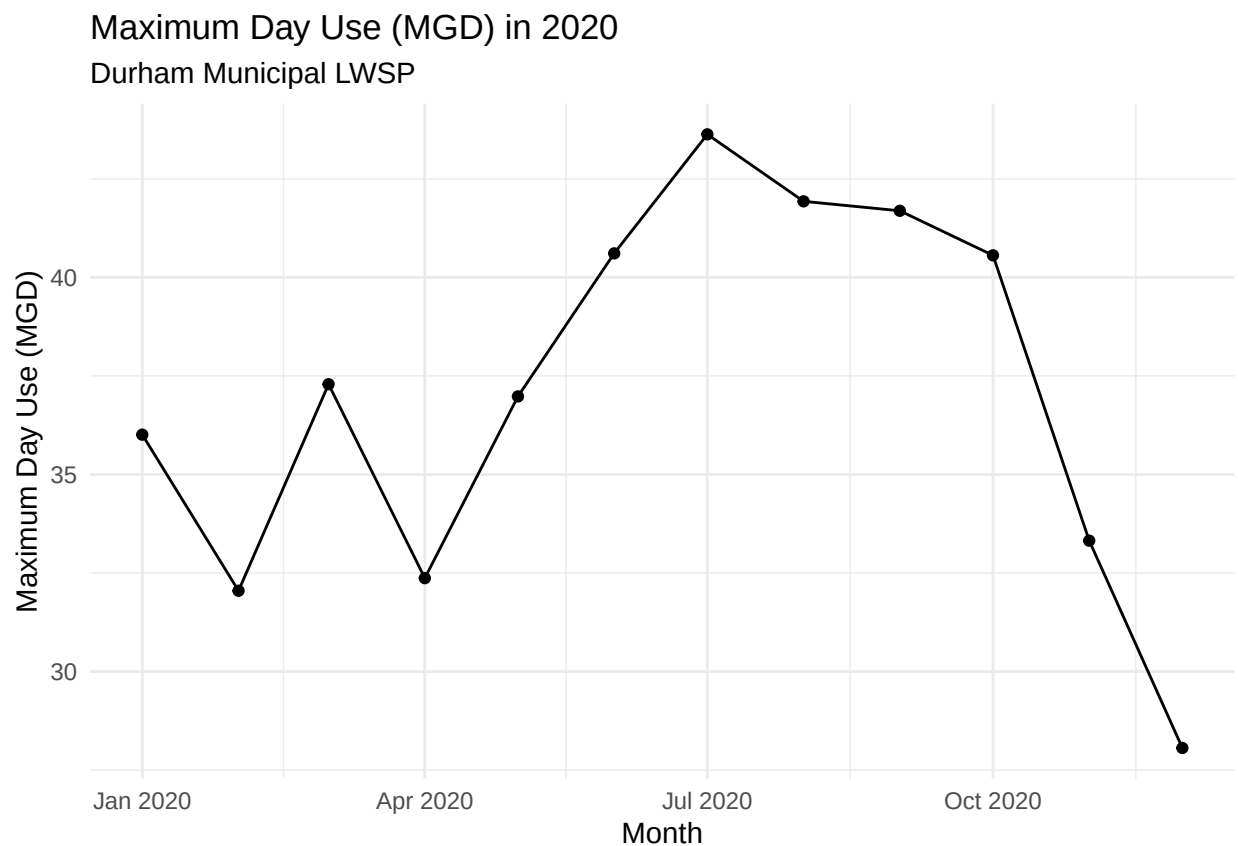
7. Use the function above to extract and plot max daily withdrawals for Durham (PWSID='03-32-010') for each month in 2020. Then show the structure of the dataframe with either the `str()` or the `glimpse()` function.

```

#7
df_durham_2020 <- scrape_it("03-32-010", 2020)

ggplot(df_durham_2020, aes(x = Date, y = Maximum_Day_Use)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Maximum Day Use (MGD) in 2020",
    subtitle = "Durham Municipal LWSP",
    x = "Month",
    y = "Maximum Day Use (MGD)") +
  theme_minimal()

```



```
glimpse(df_durham_2020)
```

```
## Rows: 12
## Columns: 7
## $ Month      <chr> "Jan", "Feb", "Mar", "Apr", "May", "June", "Jul", "Aug~
## $ Year       <dbl> 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, ~
## $ Maximum_Day_Use <dbl> 36.01, 32.05, 37.29, 32.37, 36.98, 40.61, 43.63, 41.93~
## $ Water_System <chr> "Durham", "Durham", "Durham", "Durham", "Durham", "Dur~
## $ PWSID      <chr> "03-32-010", "03-32-010", "03-32-010", "03-32-010", "0~
## $ Ownership  <chr> "Municipality", "Municipality", "Municipality", "Munic~
## $ Date       <date> 2020-01-01, 2020-02-01, 2020-03-01, 2020-04-01, 2020-~
```

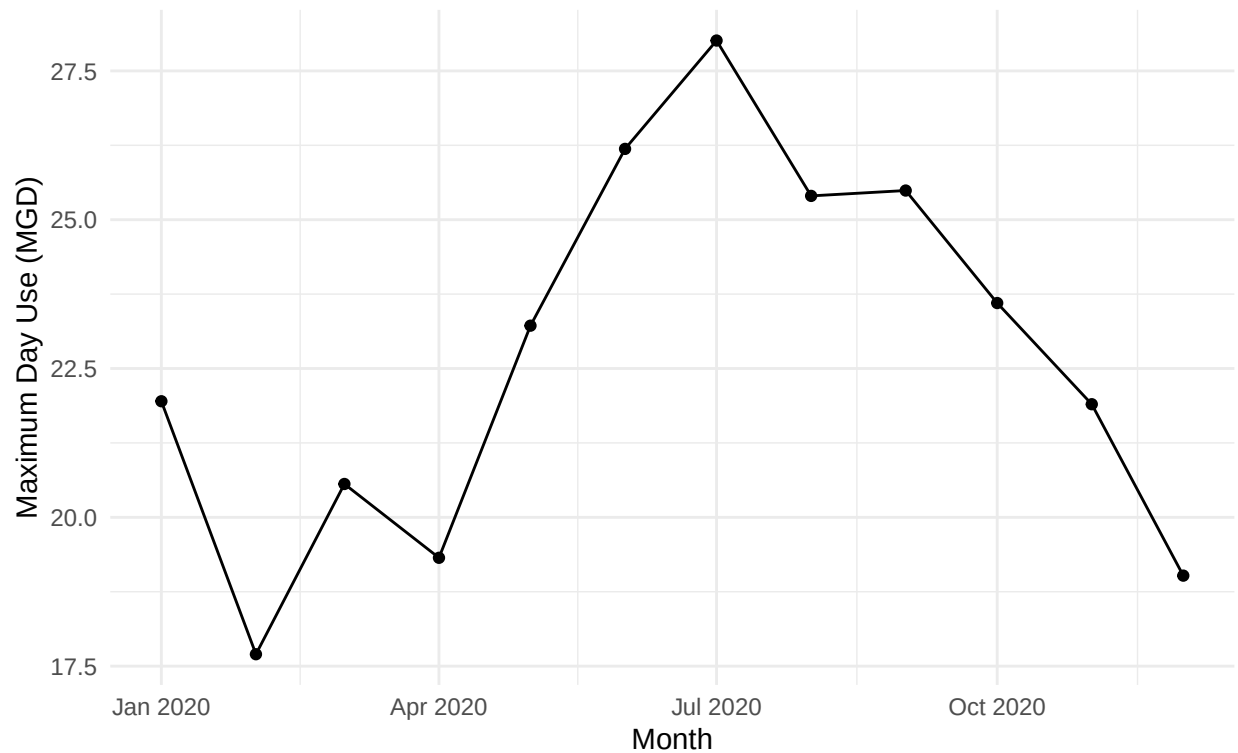
8. Use the function above to extract data for Cary (PWSID = '03-92-020') in 2020. Combine this data with the Durham data collected above and create a plot that compares Cary's to Durham's water withdrawals.

```
#8
df_cary_2020 <- scrape_it("03-92-020", 2020)

ggplot(df_cary_2020, aes(x = Date, y = Maximum_Day_Use)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Maximum Day Use (MGD) in 2020",
    subtitle = "Cary Municipal LWSP",
    x = "Month",
    y = "Maximum Day Use (MGD)") +
  theme_minimal()
```

Maximum Day Use (MGD) in 2020

Cary Municipal LWSP



```
glimpse(df_cary_2020)
```

```
## Rows: 12
## Columns: 7
## $ Month      <chr> "Jan", "Feb", "Mar", "Apr", "May", "June", "Jul", "Aug~
## $ Year       <dbl> 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, ~
## $ Maximum_Day_Use <dbl> 21.95, 17.70, 20.56, 19.32, 23.22, 26.19, 28.01, 25.40~
## $ Water_System <chr> "Cary", "Cary", "Cary", "Cary", "Cary", "Cary", "Cary"~
## $ PWSID      <chr> "03-92-020", "03-92-020", "03-92-020", "03-92-020", "0~
## $ Ownership  <chr> "Municipality", "Municipality", "Municipality", "Munic~
## $ Date       <date> 2020-01-01, 2020-02-01, 2020-03-01, 2020-04-01, 2020--
```

9. Use the code & function you created above to plot Cary's max daily withdrawal by months for the years 2018 thru 2023. Add a smoothed line to the plot (method = 'loess').

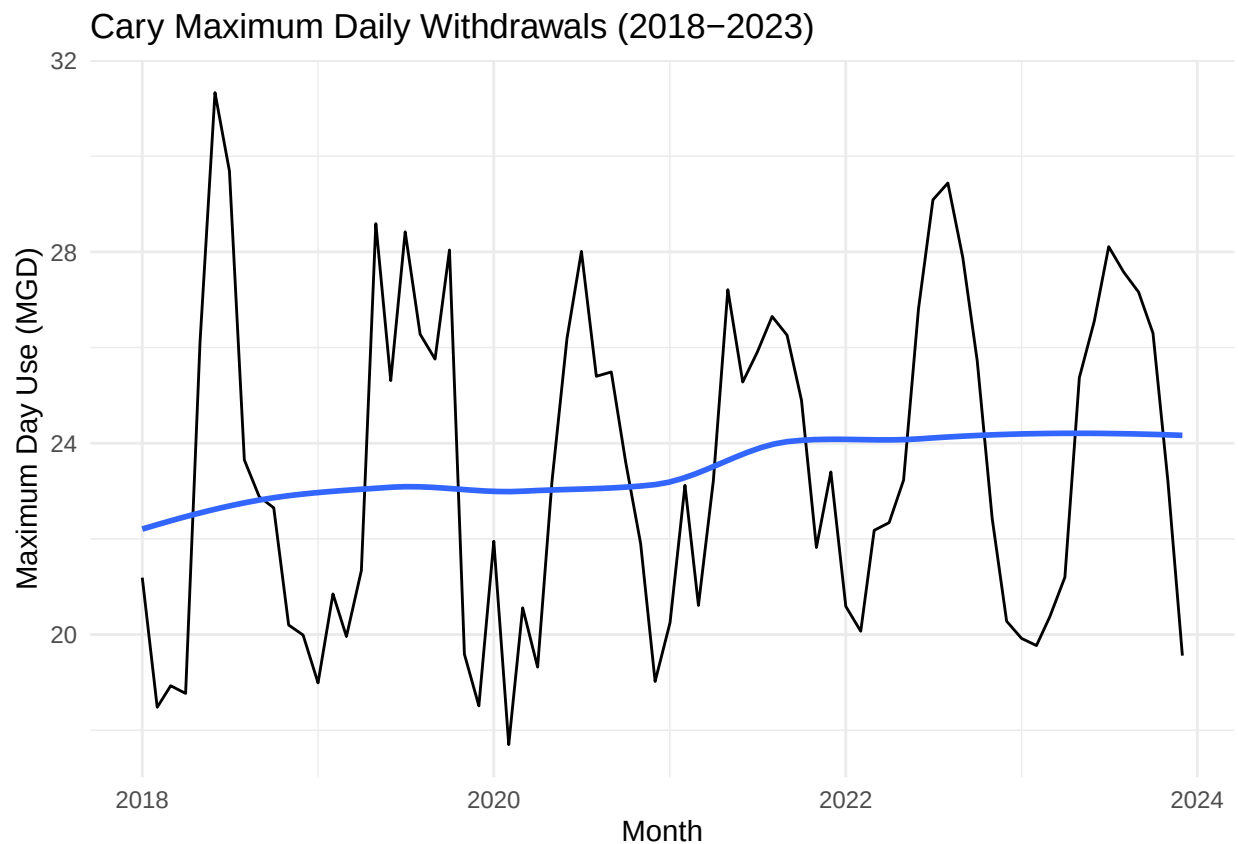
TIP: See Section 3.2 in the "10_Data_Scraping.Rmd" where we apply "map2()" to iteratively run a function over two inputs. Pipe the output of the map2() function to bind_rows() to combine the dataframes into a single one, and use that to construct your plot.

```
#9
PWSID_cary <- "03-92-020"
years <- 2018:2023

df_cary_all <- map2(PWSID_cary, years, scrape_it) |> bind_rows()
```

```
ggplot(df_cary_all, aes(x = Date, y = Maximum_Day_Use)) +
  geom_line() +
  geom_smooth(method = "loess", se = FALSE) +
  labs(
    title = "Cary Maximum Daily Withdrawals (2018-2023)",
    x = "Month",
    y = "Maximum Day Use (MGD)" ) +
  theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



Question: Just by looking at the plot (i.e. not running statistics), does Cary have a trend in water usage over time? > Answer: Cary doesn't appear to have a strong trend in water usage but a slightly upward trend indicating by the smoothed line. There is a small rise in 2021 otherwise the trend line remains fairly flat. >